Assignment 13.2

Name :N.Akhil

HT.NO : 2303A51065

B.NO : 16

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

• Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

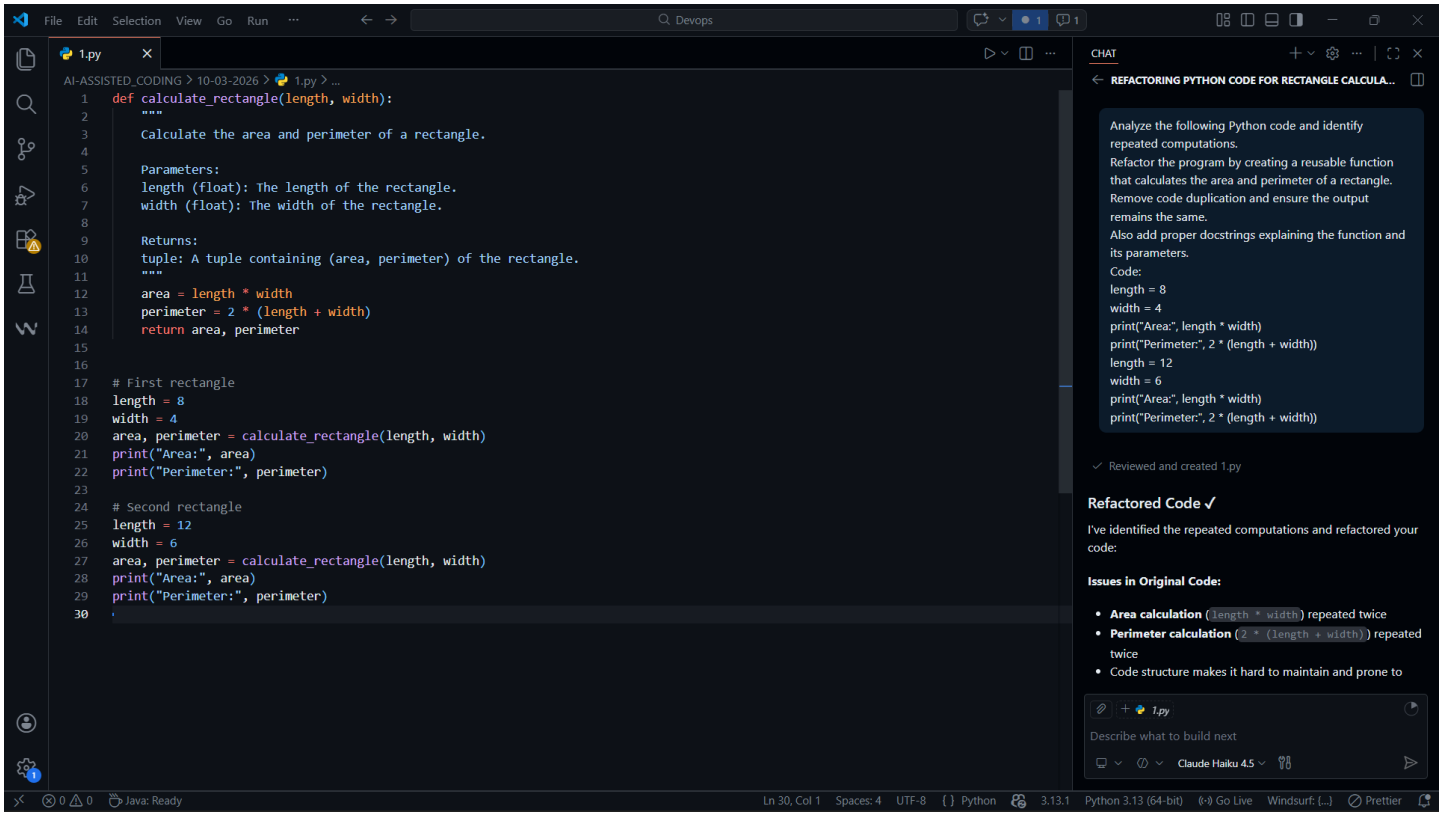
Instructions:

• Ask AI to locate repeated computations within the program.

• Encapsulate repeated logic into reusable functions.

• Verify that program output remains unchanged after refactoring.

• Add proper docstrings to all defined functions



Created a reusable calculate_rectangle() function with:

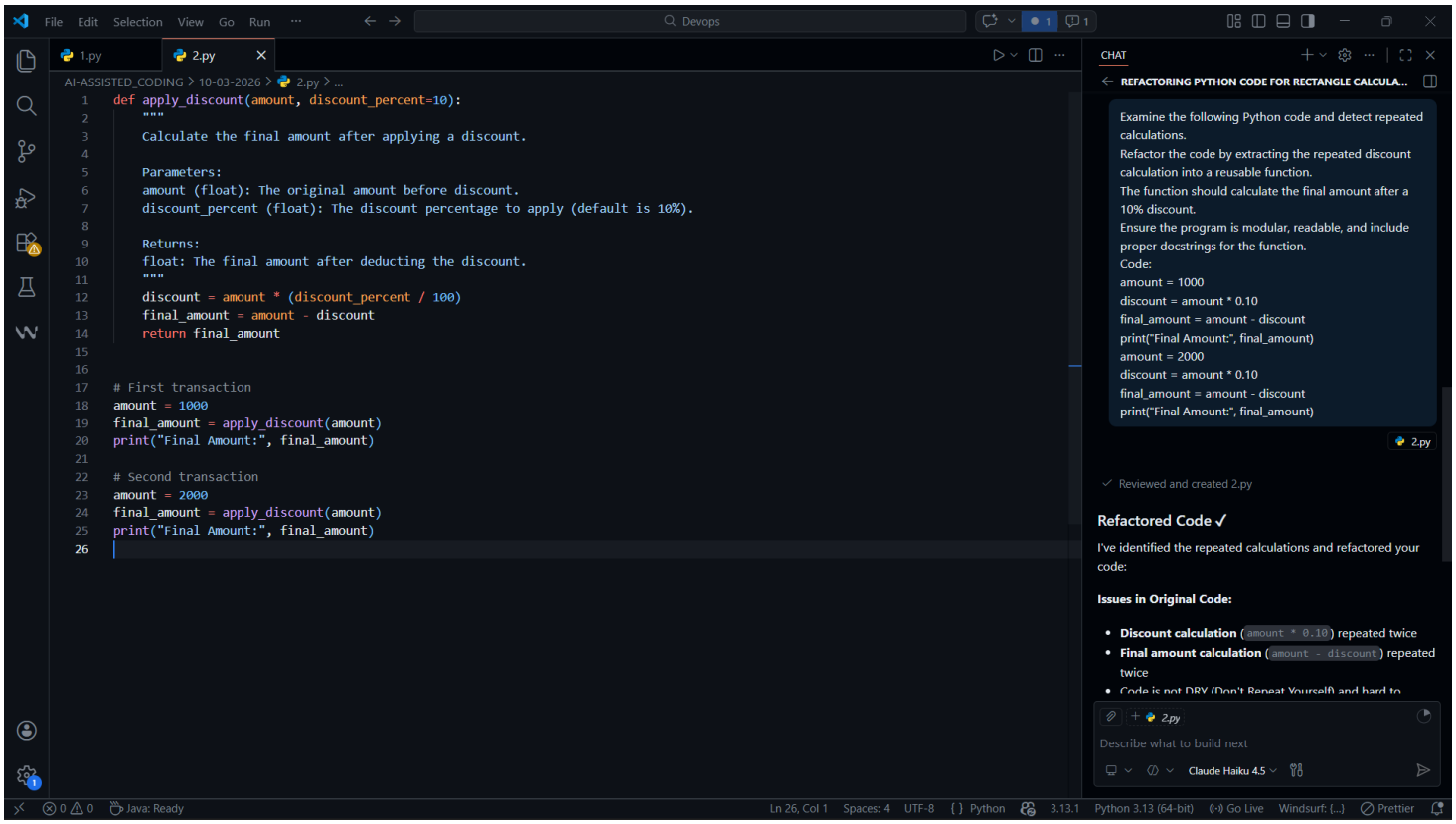
- ✓ Comprehensive docstring explaining the function, parameters, and return value
- ✓ Single source of truth for both calculations
- ✓ Returns both area and perimeter as a tuple
- ✓ Same output as original code

Task Description -2 (Refactoring – Modularizing Inline Calculations)

- **Task: Use AI to refactor a Python script by extracting repeatedinline calculations into reusableand well-documented functions.**

Instructions:

- **Identify repeated calculation patterns in the code.**
- **Convert them into reusable functions.**
- **Ensure readability and proper documentation**

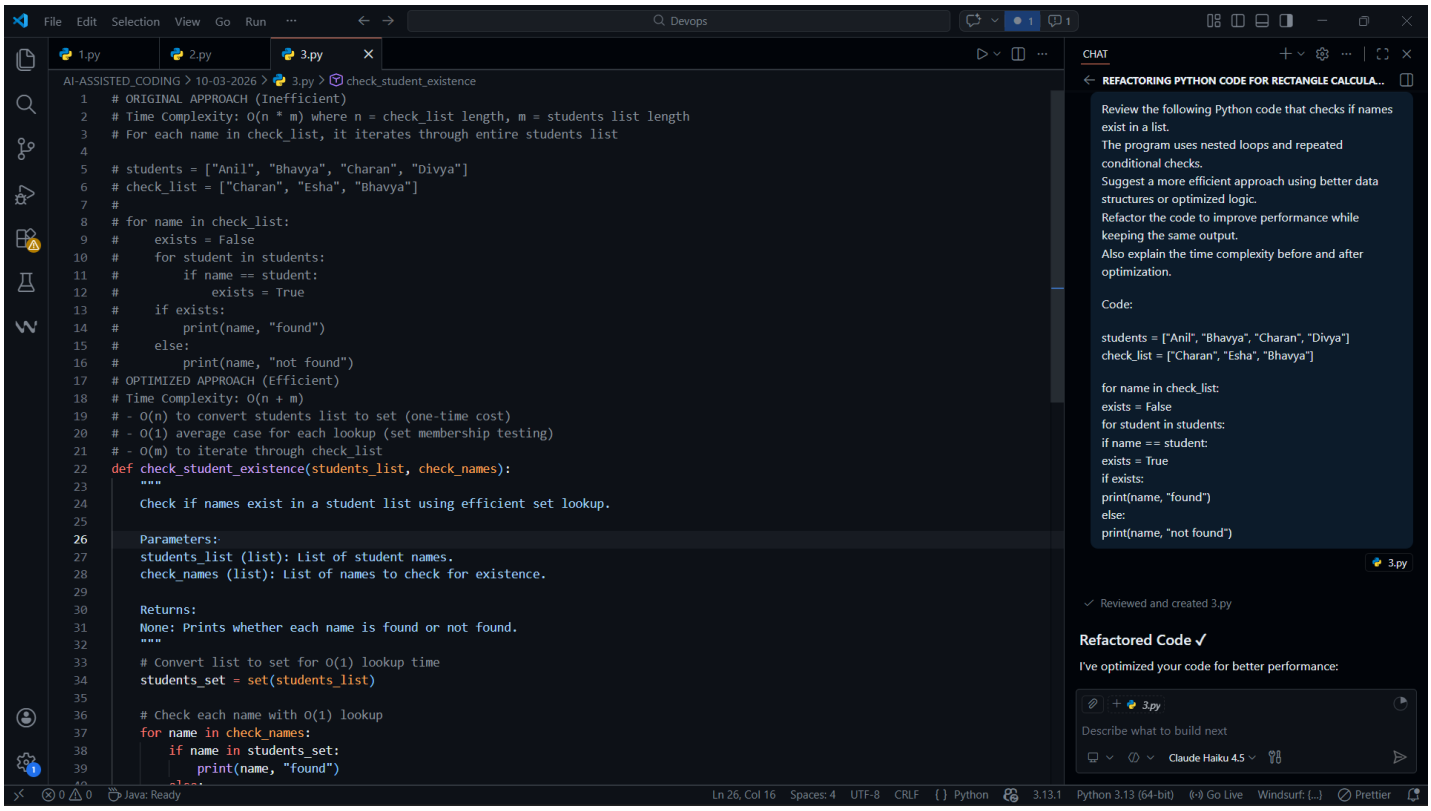


Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

- **Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.**

Instructions:

- **Use AI to recommend more efficient data structures or logic simplifications.**
- **Replace inefficient constructs while preserving program behavior.**
- **Compare execution efficiency before and after refactoring.**




```python
        else:
            print(name, "not found")


# Usage
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

check_student_existence(students, check_list)

```

Optimized Approach (Efficient):

Time Complexity: O(n + m)

O(n) to convert students list to a set (one-time cost)

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

- **Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.**

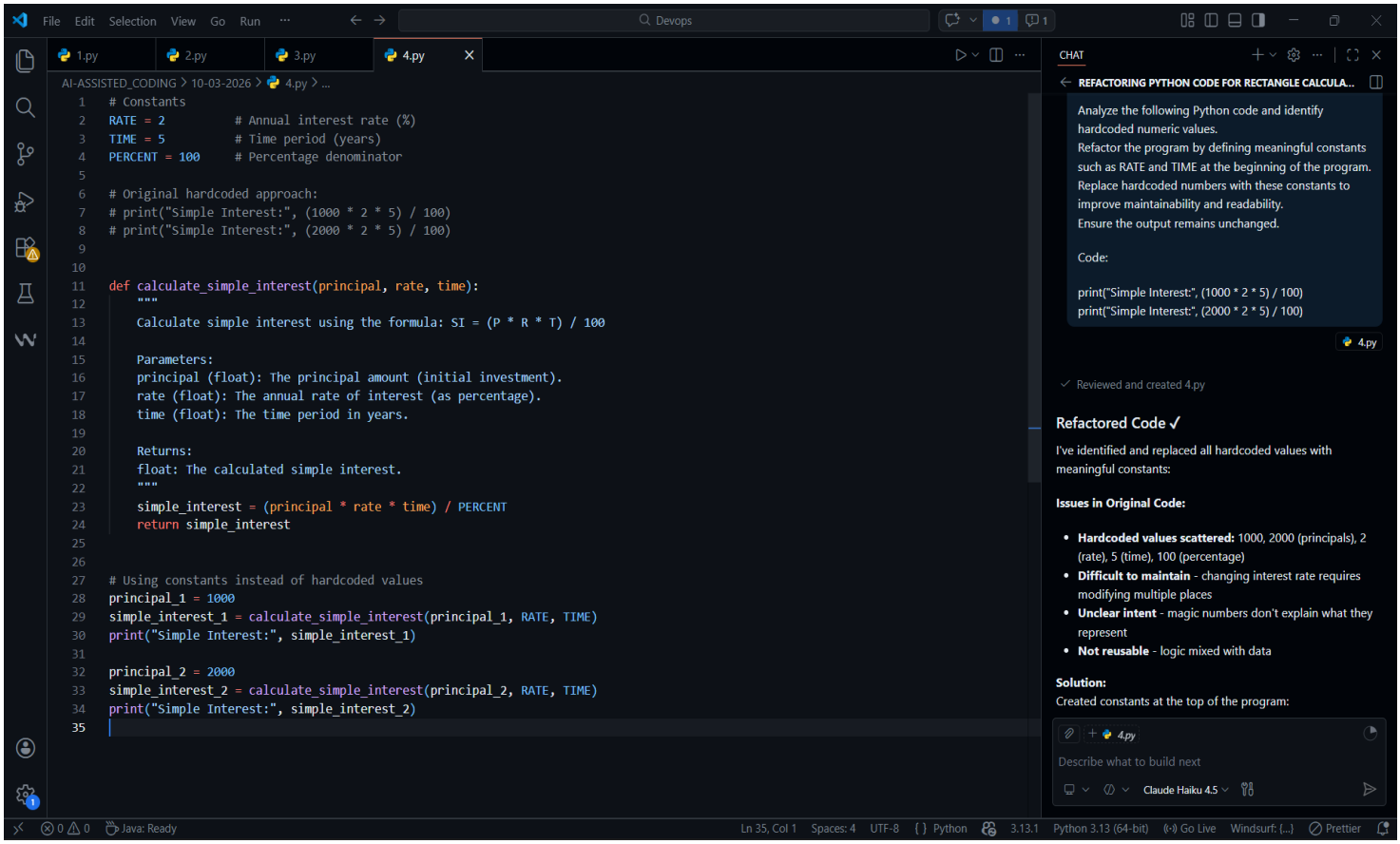
Instructions:

- **Detect hardcoded numeric values used directly in expressions.**
- **Define descriptive constants at the beginning of the program.**
- **Update calculations to use these constants without changing results.**

Sample Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)



Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

- **Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..**

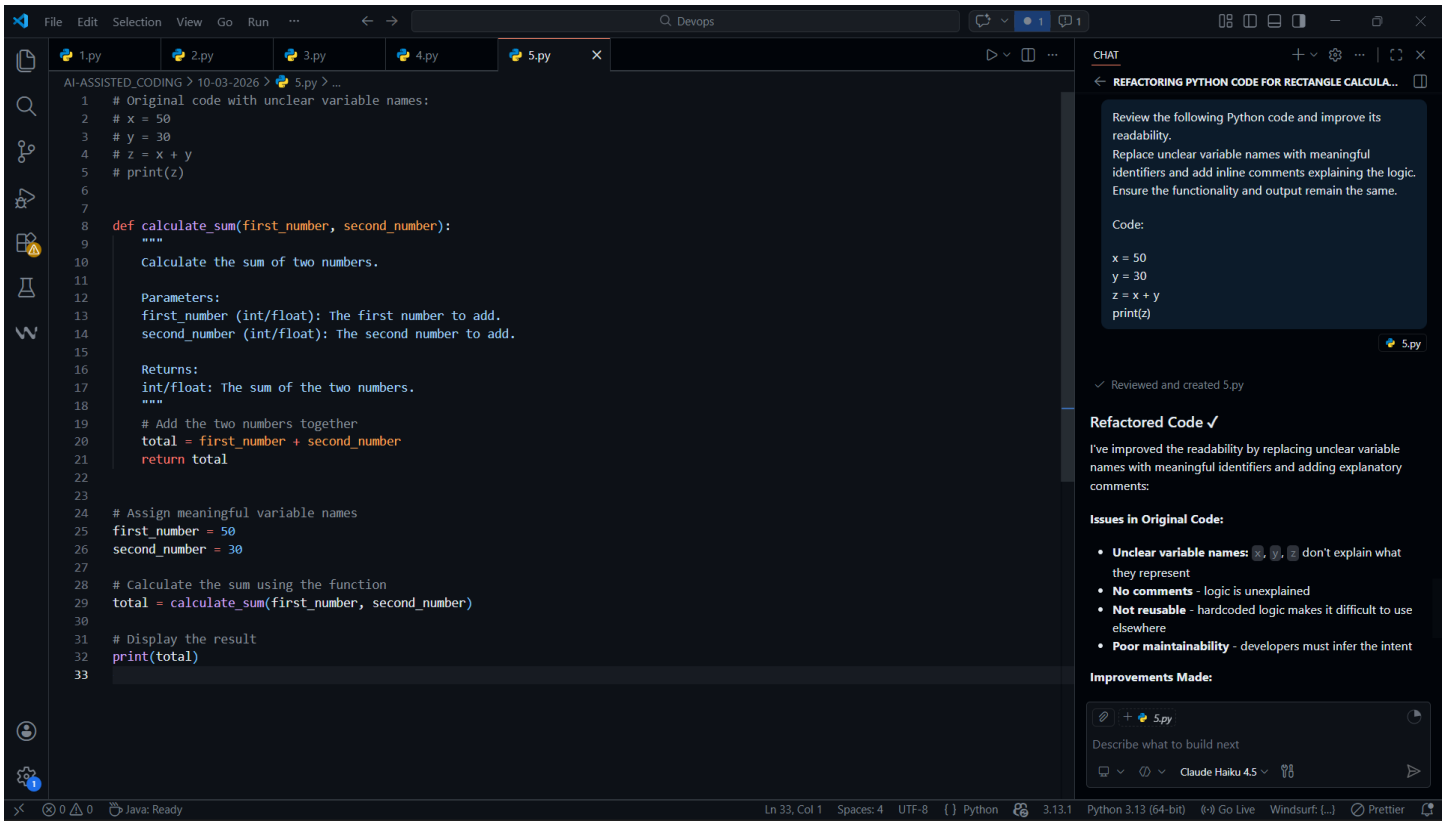
Instructions:

- **Replace non-descriptive variable names with meaningful identifiers.**
- **Add inline comments where the logic may be unclear.**
- **Ensure program output remains the same.**

Sample Code:

x = 50

y = 30

z = x + y

print(z)